# COMPTE RENDU – TP3

## Déploiement TLS, authentification mutuelle et gestion des certificats

### OBJECTIF

Mettre en place un serveur HTTPS avec Nginx, vérifier la chaîne de certification, configurer l'authentification mutuelle des clients et automatiser la surveillance et le renouvellement des certificats.

### 1. CONFIGURATION TLS DE NGINX

Le serveur `www.tp.local` a été configuré avec Nginx sur le port HTTPS 443. La configuration utilise un certificat serveur accompagné du certificat de l'autorité intermédiaire. Le certificat racine n'est pas envoyé par le serveur, car il constitue le point de confiance du client.

Voici le rôle des principales directives utilisées :

| Directive | Justification |
|---|---|
| `listen 443 ssl;` | Active l'écoute HTTPS sur le port 443. |
| `server_name www.tp.local;` | Définit le nom de domaine du serveur. |
| `ssl_certificate` | Indique le chemin du certificat serveur et de la chaîne intermédiaire. |
| `ssl_certificate_key` | Indique le chemin de la clé privée correspondant au certificat serveur. |
| `ssl_protocols TLSv1.2 TLSv1.3;` | Autorise uniquement TLS 1.2 et TLS 1.3. |
| `ssl_prefer_server_ciphers off;` | Laisse la négociation des suites de chiffrement suivre les préférences du client et les règles TLS. |
| `ssl_session_tickets off;` | Désactive les tickets de session TLS. |
| `ssl_client_certificate` | Indique l'autorité de certification utilisée pour vérifier les certificats clients. |
| `ssl_verify_client on;` | Rend obligatoire la présentation d'un certificat client. |
| `ssl_verify_depth 2;` | Limite la profondeur de vérification de la chaîne de certification. |
| `ssl_crl` | Indique la liste des certificats révoqués à vérifier. |
| `add_header Strict-Transport-Security "max-age=31536000" always;` | Active HSTS et demande au navigateur de privilégier HTTPS pendant un an. |

## 2. VÉRIFICATIONS ET TESTS TLS

Les tests effectués avec OpenSSL ont confirmé le bon fonctionnement des deux versions TLS :

- **TLS 1.2 :** négociation avec la suite `ECDHE-ECDSA-AES256-GCM-SHA384`.
- **TLS 1.3 :** négociation avec la suite `TLS_AES_256_GCM_SHA384` et le groupe `X25519`.

La vérification du certificat et du nom `www.tp.local` a réussi.
Le test Nmap effectué avec la commande :

```
nmap --script ssl-enum-ciphers -p 443 www.tp.local
```

a confirmé la disponibilité de TLS 1.2 et TLS 1.3, avec des suites de chiffrement de niveau A.

### 3. AUTHENTIFICATION MUTUELLE (mTLS)

Trois tests ont été réalisés avec Curl afin de vérifier le comportement du serveur face aux différents certificats clients.

| Test effectué | Résultat observé |
|---|---|
| Connexion sans certificat client | HTTP 400 : certificat client obligatoire. |

| | |
|---|---|
| Connexion avec le certificat révoqué d'Alice | HTTP 400 : certificat refusé. |
| Connexion avec le certificat valide de Bob | HTTP 200 : accès accepté. |

Le certificat d'Alice a été rejeté à cause de sa révocation. Celui de Bob a été accepté grâce à la présentation de sa chaîne complète de certification.

Ces résultats confirment que Nginx exige un certificat client valide et vérifie sa révocation.

## 4. SURVEILLANCE DES CERTIFICATS

Le script `surveiller.py` permet de surveiller un certificat individuel ou tous les certificats `.crt` présents dans un dossier.
Il affiche le nombre de jours restants avant expiration et utilise trois niveaux d'alerte :

- **OK :** certificat valide pendant au moins 30 jours (code retour 0).
- **ALERTE :** certificat expirant dans moins de 30 jours (code retour 1).
- **CRITIQUE :** certificat expirant dans moins de 7 jours (code retour 2).

Les tests réalisés ont donné les résultats suivants :

- Certificat serveur valide pendant 89 jours : OK, code retour 0.
- Certificat de test valable environ 5 jours : CRITIQUE, code retour 2.
- Surveillance d'un dossier contenant plusieurs certificats : fonctionnement correct.

## 5. RENOUVELLEMENT AUTOMATIQUE DES CERTIFICATS

Le script `renouveler.sh` a été vérifié avec la commande `bash -n`.

En fonctionnement normal, il indique qu'aucun renouvellement n'est nécessaire lorsque le certificat est encore suffisamment valide.

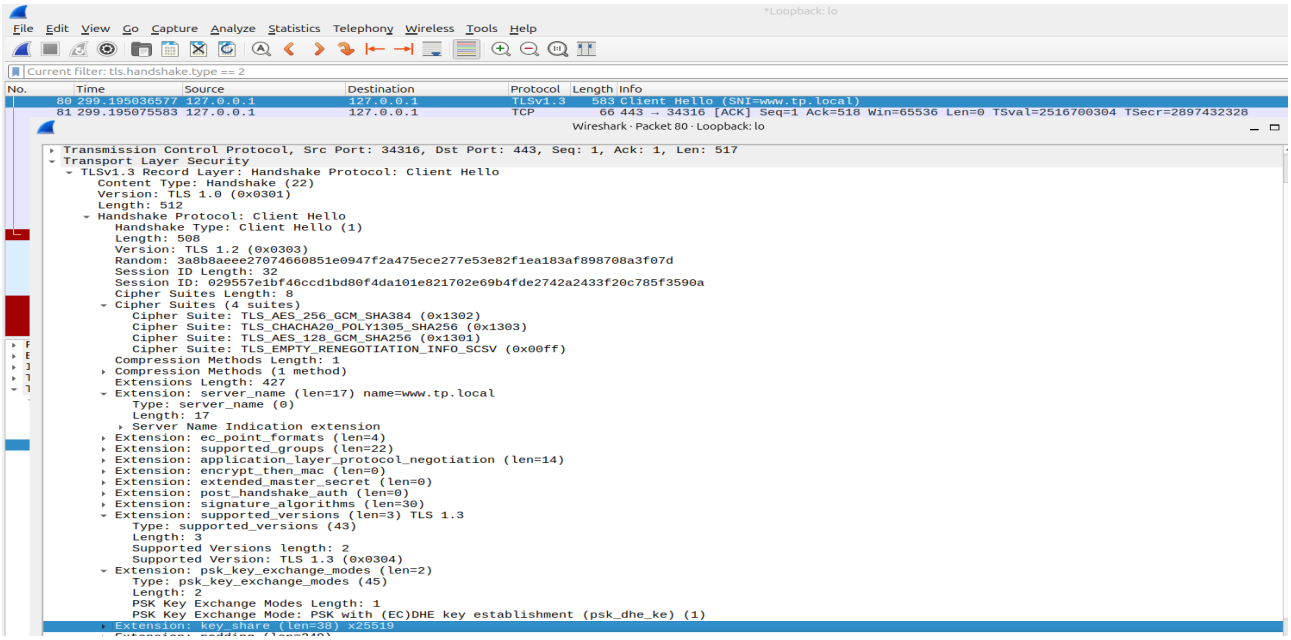
Le mode `--test` a permis de vérifier la génération d'une nouvelle clé, la création et la signature d'une demande de certificat, la vérification de la chaîne de certification ainsi que la correspondance entre la clé et le certificat. Ce test n'a entraîné aucune modification des fichiers de Nginx.

Une tâche Cron a également été configurée pour lancer automatiquement le script chaque jour à **02 h 00**. Les sorties sont enregistrées dans le fichier `/var/log/renouveler-certificat.log`.

## 6. ANALYSE DE LA NÉGOCIATION TLS AVEC WIRESHARK

Une capture Wireshark a été réalisée afin d'observer la négociation TLS entre le client et le serveur.

**Capture 1 : Message ClientHello**



### a) Message ClientHello

Le message ClientHello est envoyé par le client au début de la négociation TLS. Il contient notamment :

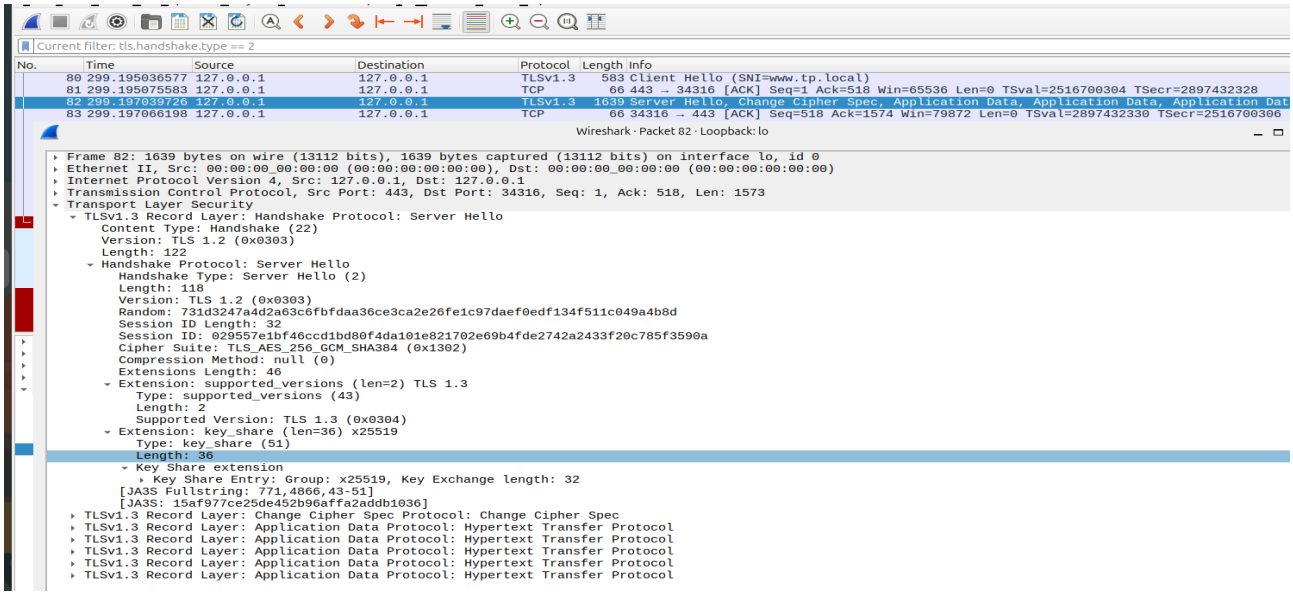
- Les versions TLS prises en charge.
- Les suites de chiffrement proposées.
- Le nom du serveur (`www.tp.local`) grâce au SNI.
- Les paramètres cryptographiques proposés.

### b) Message ServerHello

Le message ServerHello est envoyé par le serveur en réponse au ClientHello. Il permet au serveur de communiquer les paramètres retenus pour la connexion.
Dans notre capture, le serveur a sélectionné :

- Version : TLS 1.3.
- Suite de chiffrement : `TLS_AES_256_GCM_SHA384`.
- Groupe d'échange de clés : `X25519`.

**Capture 2 : Message ServerHello**




### c) Pourquoi le certificat du serveur n'est-il pas visible en TLS 1.3 ?

En TLS 1.3, les messages du handshake qui suivent ServerHello, notamment EncryptedExtensions, Certificate et CertificateVerify, sont chiffrés.

Le certificat du serveur est donc bien transmis, mais son contenu n'est pas directement lisible dans Wireshark sans disposer des clés de déchiffrement TLS.

## 7. RÉPONSES AUX QUESTIONS

À noter que les réponses aux questions se trouvent dans le README

## CONCLUSION

Ce TP a permis de mettre en place un serveur HTTPS sécurisé avec TLS 1.2 et TLS 1.3, ainsi qu'un mécanisme d'authentification mutuelle des clients. Les différents tests ont confirmé le rejet des certificats absents ou révoqués et l'acceptation d'un certificat valide. Enfin, les scripts de surveillance et de renouvellement ont permis de mettre en place les bases d'une gestion automatisée des certificats.